Казахстанско-Немецкий университет. Факультет инжиниринга и информационных технологий

Netly — серверная часть сервиса планирования личного бюджета

Самостоятельная работа студента № 1. Результаты практических работ № 1–4

Дисциплина Проектирование и разработка серверной части приложений

Исполнитель Тугельбаев Алибек; 6B06105 «Телематика»; группа 4TM

Преподаватель Самат И. М., магистр технических наук, старший преподаватель

Алматы, 28 октября 2026

Netly отвечает, сколько денег свободно в этом месяце

1 Кто пользователи

- Студенты и начинающие специалисты
- Доход из 2–4 источников
- Несколько платных подписок

2 Проблема

- Банк показывает траты постфактум
- Доход и подписки в разных циклах
- Неясно, сколько можно тратить

3 Польза

- Данные вводятся один раз
- Всё в месячном эквиваленте
- Сроки — за 7 дней до списания

Измеримо: свободный остаток и взнос по каждой цели — одним запросом, вместо ручной таблицы раз в месяц

Шесть историй покрывают путь от входа до сводки

№	Как …	я хочу …	чтобы …
US-1	пользователь с подработками	вводить доход в привычных единицах	видеть месячный эквивалент
US-2	пользователь	указать или пропустить расходы	остаток был реалистичным
US-3	владелец подписок	вести подписки и долги со сроками	их цена вошла в бюджет
US-4	копящий пользователь	создавать цели и пополнять их	знать ежемесячный взнос
US-5	пользователь	видеть сводку и ближайшие сроки	не считать вручную
US-6	пользователь	входить по email и паролю	мои данные видел только я

Реализованы на сервере: US-3, US-4, US-6. US-1, US-2, US-5 — после ПР № 5–7

Критерии приёмки двух историй проверяются автотестами

US-3. Подписка в месячном эквиваленте

Given подписка Netflix стоит 119.88 в год

When `POST /api/v1/subscriptions`

Then 201 Created и monthlyCost = 9.99

Тест: `SubscriptionsAPITests`

US-6. Вход только по верному паролю

Given есть аккаунт alibek@netly.kz

When вход с неверным паролем

Then 401, тот же ответ, что для чужого email

Тест: `AccountTests`

Всего 73 автотестов в двух сервисах, все проходят

Swift на сервере: один язык с iOS-клиентом

Критерий	Swift: Vapor / Hummingbird	Node.js + Express	Python + FastAPI
Язык iOS-клиента	**да, общие Codable-модели**	нет	нет
Типы при компиляции	**строгие, Swift 6**	только с TypeScript	подсказки типов
Асинхронность	**async/await на SwiftNIO**	event loop	asyncio
ORM, JWT, миграции	Fluent, JWTKit, migrations	Prisma, jsonwebtoken	SQLAlchemy, Alembic
Мой опыт	**пишу iOS на Swift**	базовый	базовый

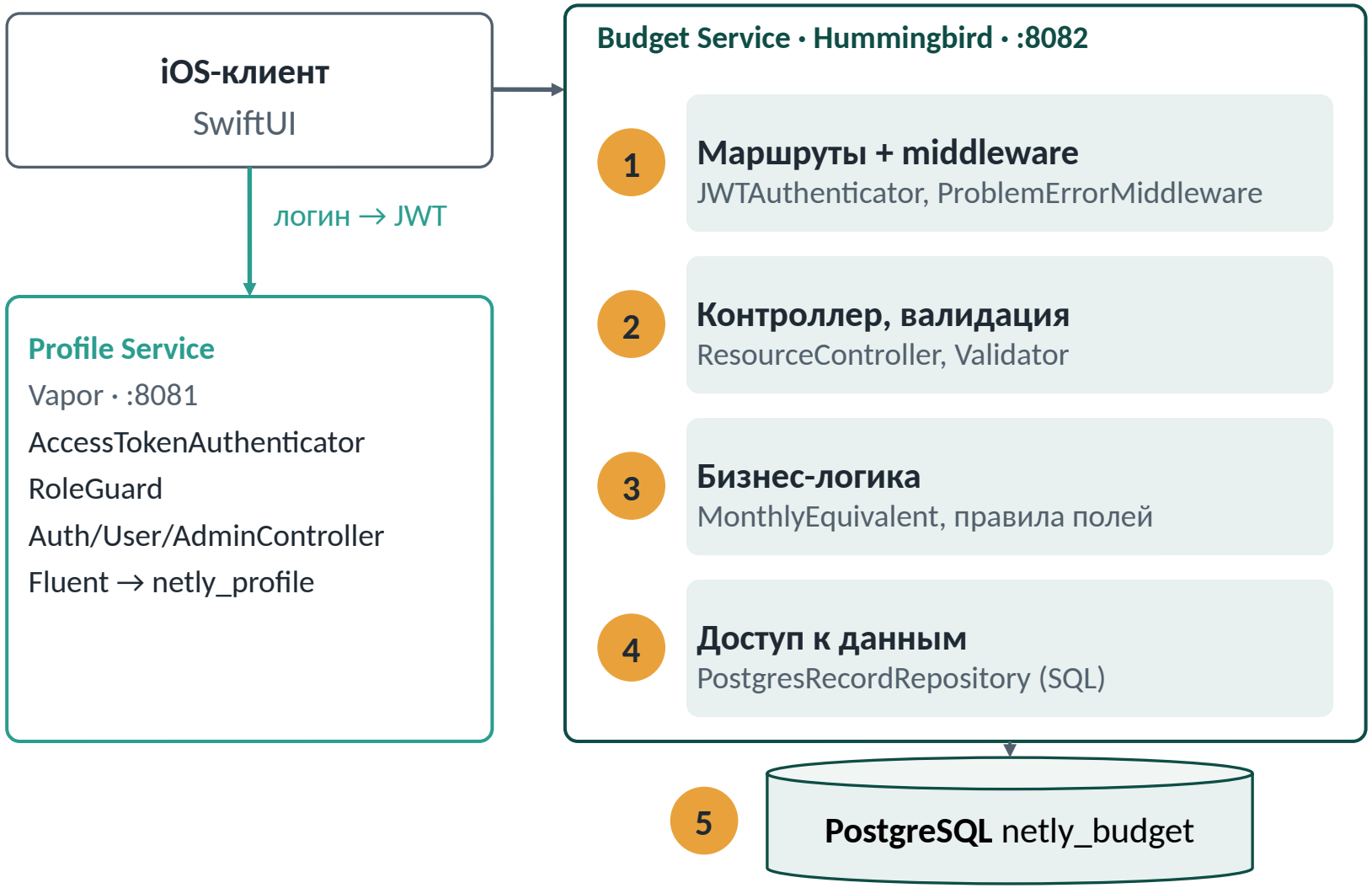
Profile Service → Vapor

связанные данные, вход, ORM Fluent и JWT из коробки

Budget Service → Hummingbird

плоская модель и расчёты, минимум зависимостей

Запрос проходит четыре слоя до базы данных



`POST /api/v1/goals`

1 токен → user_id, иначе 401

2 JSON → поля, иначе 422

3 расчёт месячного взноса

4 INSERT … с user_id

5 строка из БД → 201 + Location

Profile ← Budget: запрос дохода — план

Секреты и адреса БД — только в переменных окружения

```
netly/
├── docs/api/      OpenAPI-контракты
├── scripts/       demo, dev-token
├── services/
│   ├── profile-service/   Vapor
│   │   └── Sources/App/
│   └── budget-service/    Hummingbird
│       ├── Sources/App/
│       └── Tests/AppTests/
├── .env.example
└── docker-compose.yml
```

```
# .env.example
DB_HOST=localhost
DB_PORT=5432
DB_NAME=netly_budget
DB_USER=netly
DB_PASSWORD=netly
JWT_SECRET=<≥ 32 байт>
PORT=8082
```

- .env в .gitignore — пароли не в git
- Одна сборка для dev и prod
- Нет DB_* или секрет короче 32 байт — сервис не стартует

Внутри базы — FK, между сервисами — user_id из JWT

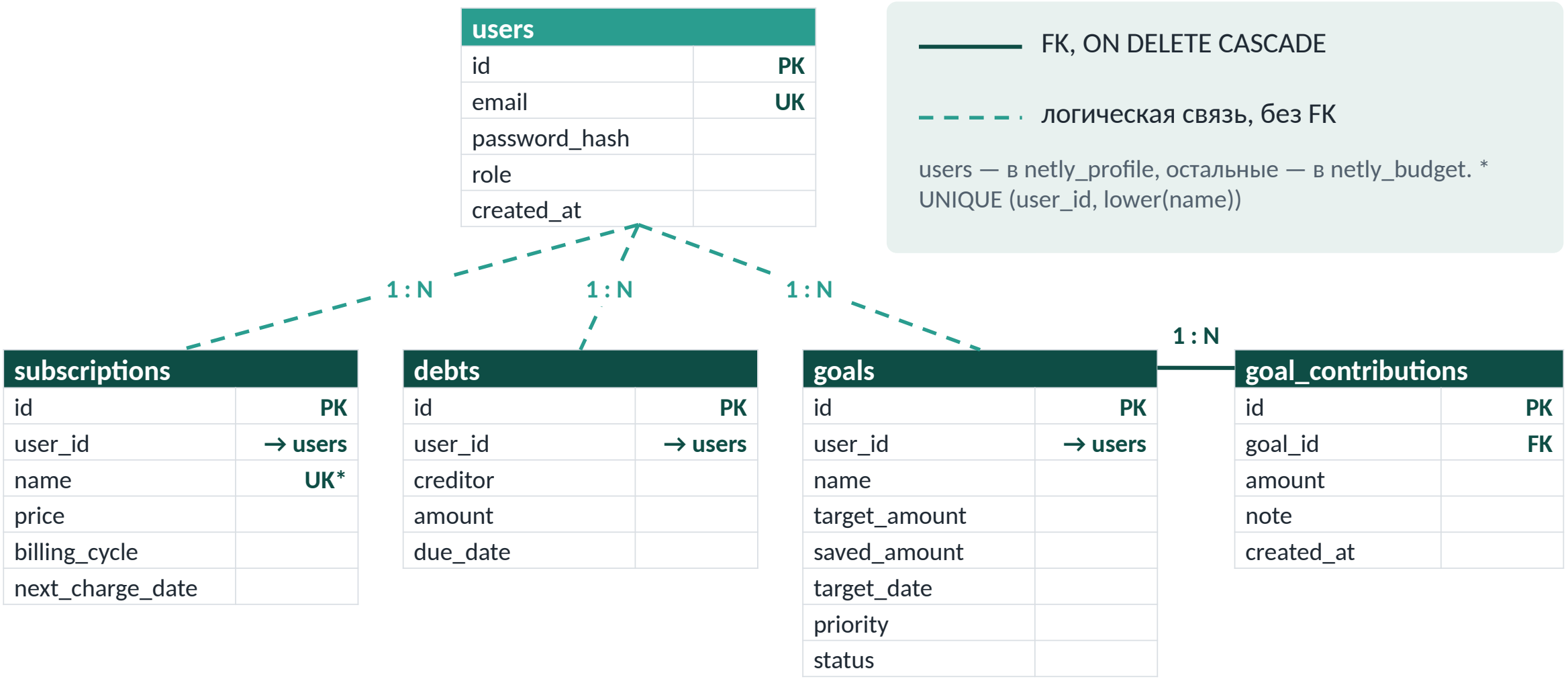


Схема растёт миграциями, каждый индекс — под свой запрос

Миграции (откатываются)

budget 001 `create_budget_schema`
4 таблицы, CHECK, FK

↓

budget 002 `add_goal_status_index`
индекс под фильтр статуса

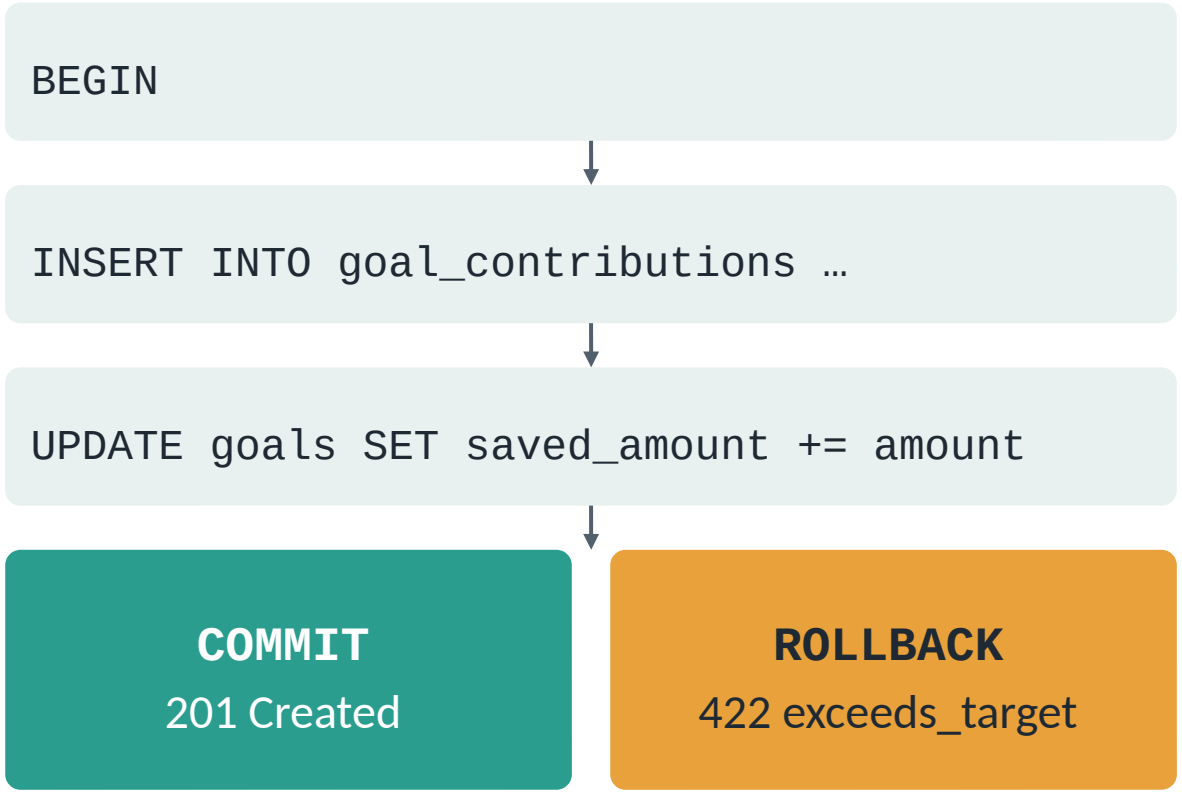
↓

profile 001 `create_users`
users, UNIQUE email

Индекс	Ускоряет запрос
`goals (user_id, status, created_at, id)`	GET /goals?status=active
`<таблица> (user_id, created_at, id)`	списки с пагинацией
`subscriptions (user_id, lower(name))`	UNIQUE: дубль → 409
`goal_contributions (goal_id, …)`	история цели, каскад FK
`users (email)`	поиск при входе, UNIQUE

Порядок сортировки входит в индекс — страница читается без сортировки

Пополнение цели атомарно: две записи или ни одной



CHECK saved_amount ≤ target_amount нарушен — откатывается и INSERT

Ограничения целостности

- **NOT NULL** на всех полях, кроме note
- **CHECK**: суммы > 0, priority 1–10, enum
- **NUMERIC(12,2)** для денег, не float
- **FK** goal_id → goals, ON DELETE CASCADE
- **UNIQUE** email; имя подписки у пользователя
- **role IN ('user', 'admin')**

Ресурс goals: пять CRUD-операций и вложенные пополнения

Метод	Путь	Назначение	Успех
POST	`/api/v1/goals`	создать цель	201
GET	`/api/v1/goals`	список, фильтр status	200
GET	`/api/v1/goals/{id}`	одна цель	200
PATCH	`/api/v1/goals/{id}`	частично изменить	200
DELETE	`/api/v1/goals/{id}`	удалить	204
POST	`/api/v1/goals/{id}/contributions`	пополнить (транзакция)	201
GET	`/api/v1/goals/{id}/contributions`	история пополнений	200

subscriptions и debts — та же пятёрка. Всего 18 операций в Budget и 5 в Profile

Контракт формальный: OpenAPI 3.1, проверен линтером

```yaml
post:
  operationId: createGoalContribution
  requestBody:
    required: true
    content:
      application/json:
        schema: { $ref: …/GoalContributionCreate }
  responses:
    "201": { description: The contribution … }
    "401": { $ref: …/responses/Unauthorized }
    "404": { $ref: …/responses/NotFound }
    "422": { $ref: …/responses/ValidationFailed }
```

POST /api/v1/goals/{id}/contributions; … — сокращено, 400 опущен

- Два файла: budget- и profile-service.openapi.yaml
- У каждой операции — все коды ответа
- Примеры — реальные ответы сервиса
- Redocly lint: ошибок нет
- Схема ошибки Problem — общая для двух сервисов

Все ошибки — в одном формате RFC 9457 problem+json

Поле	Правило
name, creditor	1–100 символов после trim
суммы	> 0, не больше 2 знаков
даты	YYYY-MM-DD; сроки — в будущем
priority	целое 1–10
email, password	формат email; 8–72 байта

201 — корректный

422 — правила

400 — не JSON

409 — дубликат

Ошибки всех полей приходят разом

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json
{
  "code": "validation_failed",
  "detail": "4 fields are invalid",
  "errors": [
    {"field": "name", "code": "blank"},
    {"field": "priority",
     "code": "out_of_range"}, …],
  "instance": "/api/v1/goals",
  "status": 422,
  "title": "Validation failed" }
```

Списки отдаются страницами: limit ≤ 100 и total

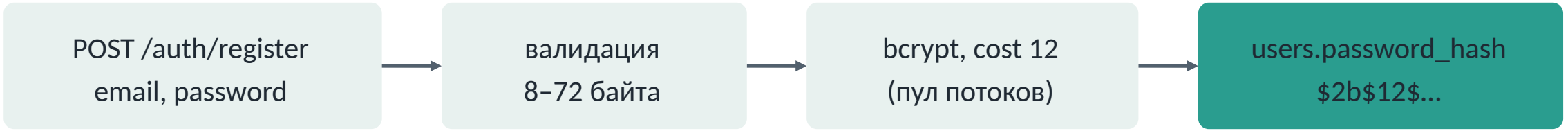
Параметр	Значения	По умолч.
`limit`	1–100	50
`offset`	≥ 0	0
`status`	active | completed | archived	—

- Порядок стабильный: created_at, id
- Только записи владельца токена
- limit=500 → 422 out_of_range

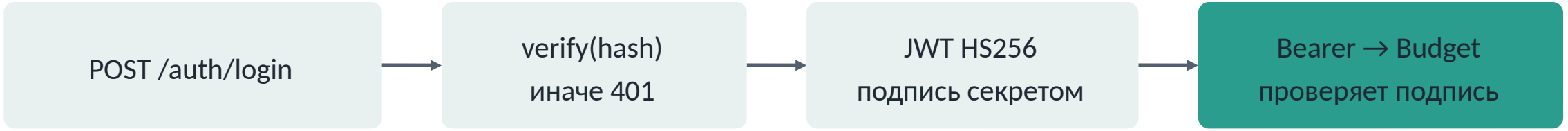
```
GET /api/v1/goals?status=active&limit=1
HTTP/1.1 200 OK
{
  "items": [{
    "name": "Laptop", "priority": 8,
    "requiredMonthlyContribution": 100,
    "status": "active",
    "targetAmount": 1500, …
  }],
  "limit": 1, "offset": 0,
  "total": 2
}
```

Пароль хранится как bcrypt-хеш, доступ — по JWT на 15 минут

Регистрация



Вход




```
// payload токена
{ "sub":  "<user id>",
  "role": "user",
  "iat":  1793209067,
  "exp":  iat + 900 }
```

- Неверный пароль и чужой email — один ответ 401
- Секрет ≥ 32 байт, общий для двух сервисов

Права проверяются до контроллера, владение — в SQL

Операция	Гость	user	admin
регистрация, вход	да	да	да
GET /users/me	401	да	да
CRUD своих записей бюджета	401	да	да
чужая запись бюджета	401	404	404
GET /admin/users	401	403	да

admin не видит чужие финансы: это не его функция. Роль admin выдаёт только команда App make-admin

Где проверяется

```
// configure.swift
let authenticated = api
  .grouped(AccessTokenAuthenticator())
authenticated
  .grouped(RoleGuard(.admin))
  .register(collection: Admin…)
```

- Middleware группы маршрутов: 401 / 403
- SQL: `WHERE user_id = $2`

Пять угроз OWASP API Top 10: мера и тест на каждую

Угроза (OWASP 2023)	Мера защиты	Как проверено
API1 BOLA: чужой id в URL	WHERE user_id из токена → 404	`showHidesOtherUsersRecords`
API2 Broken Authentication	bcrypt 12, JWT 15 мин, секрет ≥ 32 байт	`expiredTokenIsUnauthorized`
API3 Mass assignment роли	role и user_id из тела игнорируются	`roleInRegistrationBodyIsIgnored`
API4 Неограниченные ресурсы	limit ≤ 100; лимит тела запроса	`listValidatesPaging`
API5 BFLA: чужая функция	RoleGuard(.admin) → 403	`userRoleIsForbiddenFromAdminRoutes`

План: rate limit на /auth/login (API4), refresh-токены и отзыв (API2)

Сервис отвечает кодами из контракта: 201, 422, 403, 404

Успех: цель создана с токеном из Profile

```
$ curl -X POST :8082/api/v1/goals \
    -H 'Authorization: Bearer <JWT>' -d '{…}'
HTTP/1.1 201 Created
Location: /api/v1/goals/67616429-80c0-…
{"name": "Laptop", "priority": 8,
 "requiredMonthlyContribution": 100,
 "status": "active", "targetAmount": 1500,
 "targetDate": "2027-12-31", …}
```

Ошибка валидации: все поля разом

```
$ curl -X POST :8082/api/v1/goals -H '…' \
    -d '{"name": " ", "priority": 11, …}'
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json
{"code": "validation_failed",
 "detail": "4 fields are invalid",
 "errors": [{"field": "name", "code": "blank"},
  {"field": "targetAmount", …}, …]}
```

Отказ в доступе: роль user

```
$ curl :8081/api/v1/admin/users \
    -H 'Authorization: Bearer <JWT, role user>'
HTTP/1.1 403 Forbidden
Content-Type: application/problem+json
{"code": "forbidden",
 "detail": "Requires the admin role",
 "instance": "/api/v1/admin/users",
 "status": 403, "title": "Forbidden"}
```

Чужая цель: 404, как несуществующая

```
$ curl :8082/api/v1/goals/67616429-80c0-… \
    -H 'Authorization: Bearer <JWT другого>'
HTTP/1.1 404 Not Found
Content-Type: application/problem+json
{"code": "not_found",
 "detail": "Goal 67616429-… was not found",
 "instance": "/api/v1/goals/67616429-…",
 "status": 404, "title": "Not found"}
```

ПР № 1–4 выполнены, дальше — расчёт остатка

23
операций API

73
автотестов

3
миграции

Сделано

- ПР 1: два сервиса, /health, Docker Compose
- ПР 2: CRUD, problem+json, пагинация
- ПР 3: PostgreSQL, миграции, транзакция
- ПР 4: регистрация, bcrypt, JWT, роли

До СРС № 2 (ПР № 5–7)

- доходы и расходы в Profile Service
- расчёт свободного остатка по целям
- Spending Service и карта трат

Риски и ограничения

- Budget зависит от Profile синхронно
- нет rate limit на вход
- нет отзыва токена до exp

Проект — одно целое: от истории до теста

1

Архитектура

два сервиса, слои, один стек Swift

2

Данные

миграции, индексы под запросы, транзакция

3

API и защита

OpenAPI, problem+json, bcrypt, JWT, роли

Спасибо! Готов ответить на вопросы

github.com/albtugel/Netly · ветка main, коммит 86b0ed7